DEEP LEARNING LAB

Assignment 5: Vanilla RNN Seq2Seq
Long-Term Dependencies, BPTT, Gradient Stability

Indian Institute of Technology Kharagpur
Department of Artificial Intelligence

Duration: 3 Hours Total Marks: 50

Learning Objectives

By the end of this assignment, you should be able to:

- Build an encoder–decoder translation system using explicitly unrolled vanilla RNN operations.
- Understand how padding and sequence lengths affect recurrent computation.
- Simulate and observe vanishing/exploding gradients through BPTT.
- Measure recurrent gradient norms during real model training.
- Use orthogonal initialization and gradient clipping together to stabilize a model that would otherwise diverge.
- Evaluate whether a fixed-size Seq2Seq bottleneck becomes less effective for longer sequences.

Dataset and Global Settings

Three JSONL files are provided: `train.jsonl`, `val.jsonl`, `test.jsonl`.
Each record contains:

- `src`: source English sentence.
- `tgt`: target Rovelic sentence.
- `n_clauses`: number of clauses.
- `length_bucket`: short, medium, or long.

The supplied splits contain: 6000 training examples; 800 validation examples; 800 test examples.

Parameter	Value
MAX_LEN	50
BATCH_SIZE	64
EMBED_DIM	64
HIDDEN_DIM	128
LEARNING_RATE	0.001
TEACHER_FORCING_RATIO	0.5
CLIP_MAX_NORM	1.0
Seed	42

Use: `<pad>:0`, `<sos>:1`, `<eos>:2`, `<unk>:3`

Important:

- For Tasks 1–6, do not use PyTorch's built-in `nn.RNN`, `nn.LSTM`, or `nn.GRU` for the encoder or decoder.
- The recurrent computation must be explicitly unrolled using embeddings, `nn.Linear`, tensor operations, and a timestep loop.
- You will build the `nn.RNN` version later, in Task 7, on purpose.

Task 1: Dataset Loading and Inspection [3 Marks]

1.1 Dataset Ingestion [2 Marks]

- Implement `load_jsonl(path)`.
- Load all three splits.
- Extract `src`, `tgt`, and `length_bucket`.

1.2 Sanity Inspection [1 Mark]

Print:

- Number of examples in each split.
- One source–target pair.
- Distribution of the three length buckets.

Task 2: Preprocessing and Length-Aware Pipeline [9 Marks]

2.1 Vocabulary and Numericalization [3 Marks]

- Whitespace-tokenize the text.
- Construct source and target vocabularies using training data only.
- Map OOV validation/test tokens to `<unk>`.
- Add `<sos>` and `<eos>`.
- Pad/truncate to MAX_LEN=50.

2.2 Mini-Batch Generation [3 Marks]

Implement `get_batches` returning:

- Source tensor.
- Target tensor.
- True source lengths.
- True target lengths.
- Length-bucket labels.

2.3 Padding Mask & Length-Aware Recurrence [3 Marks]

- Construct a boolean mask indicating which positions correspond to real tokens rather than padding.
- For sample i, with true source length L_i, define:

$$m_t^{(i)} = \begin{cases} 1, & t < L_i \\ 0, & t \geq L_i \end{cases}$$

- Here, t is the timestep (token position) in the padded sequence.

- L_i is the true (unpadded) length of sample i.
- $m_t^{(i)} \in \{0, 1\}$ is the mask value at that position:
  - 1 if position t is a real token;
  - 0 if position t is `<pad>`.
- The mask will be used in Task 4 so that padded positions do not alter the hidden state.
- MAX_LEN=50 is only an upper bound; most sentences are shorter.
- If the encoder updates its hidden state through the `<pad>` steps after the real sentence ends, it will fail to learn even short sentences.
- Use the mask from Task 4.1 to prevent this.

Task 3: BPTT Mathematics [4 Marks]

3.1 Scalar BPTT Simulation [2 Marks]

- Implement the pure NumPy function `simulate_bptt(w, seq_length=50)` for:

$$g_{t-1} = wg_t$$

- g_t is the gradient magnitude at timestep t.
- w is the scalar recurrent weight.
- t counts down from the last timestep (50) to the first (1).
- Start from $g_{50} = 1$.
- Evaluate the simulation for: $w = 0.92$; $w = 1.08$.

3.2 Gradient Trajectory [2 Marks]

- Plot both trajectories on one figure.
- Use a logarithmic y-axis.
- Label the axes.
- Label the curves.

Task 4: Custom Masked Seq2Seq Architecture [17 Marks]

4.1 Masked Unrolled Encoder [5 Marks]

- Implement `EncoderRNN` without `nn.RNN`.
- Use:

$$h'_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$$

- At timestep t:
  - x_t is the input embedding vector (the embedded source token);
  - h_{t-1} is the hidden state from the previous timestep;
  - W_{xh} is the input-to-hidden weight matrix;
  - W_{hh} is the hidden-to-hidden (recurrent) weight matrix;

  - b is the bias vector;
  - h'_t is the candidate new hidden state before masking is applied.

- Then apply the mask:
$$h_t = m_t h'_t + (1 - m_t) h_{t-1}$$

- m_t is the mask value from Task 2.3.
- h_t is the actual hidden state used going forward.
- If $m_t = 1$ (real token), the hidden state becomes the newly computed h'_t.
- If $m_t = 0$ (`<pad>`), the hidden state simply copies h_{t-1} forward unchanged.
- After a sequence reaches its true end, its hidden state must remain unchanged during padded positions.
- Return: The final hidden state. Optionally, the complete hidden-state sequence for diagnostics.

4.2 Autoregressive Decoder [5 Marks]

Implement `DecoderRNN` using the same unrolled recurrent mechanism:

$$h_t^{\text{dec}} = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$$

Here:

- x_t is now the embedded previous target token;
- under teacher forcing, x_t comes from the ground-truth previous target token;
- otherwise, x_t comes from the decoder's own last prediction;
- h_{t-1} is the decoder's previous hidden state;
- the decoder's initial hidden state is the encoder's final hidden state;
- W_{xh}, W_{hh}, and b are the decoder's own parameters and are separate from the encoder's parameters.
- Add a linear projection from hidden state to target vocabulary logits.
- Use the encoder's final hidden state as the decoder's initial state.

4.3 Teacher Forcing vs. Free Running [7 Marks]

- Implement teacher forcing with ratio 0.5.
- Implement inference without teacher forcing.
- During inference, the decoder feeds its own previous prediction into the next timestep.
- For one validation example, show that the two modes use different decoder inputs after the first timestep.

Task 5: Gradient Tracking and Stabilization [9 Marks]

5.1 Capturing the Heartbeat [6 Marks]

- Construct a model whose encoder recurrent weight is deliberately initialized with a wide range.
- After building your Task 4.1 `EncoderRNN`, overwrite its recurrent weight using:
$$W_{hh} \sim \mathcal{U}(-1, 1)$$

- Train this model for 3 epochs with no gradient clipping.
- Use `CrossEntropyLoss` with `ignore_index=0`.
- Use Adam with learning rate 0.001.
- Immediately after `loss.backward()`, store:

$$\|\nabla W_{hh}\|_2$$

- This is the L2 norm of the loss gradient with respect to the recurrent weight matrix.
- In PyTorch, this corresponds to `W_hh.weight.grad.norm()`.
- Plot the gradient norm on a logarithmic scale over training iterations.
- You should observe large, chaotic spikes.
- Default PyTorch initialization keeps W_{hh}'s spectral radius below 1, so it will not explode here.
- The wide re-initialization above is what forces genuine exploding gradients.

5.2 Stabilization with Orthogonal Init & Gradient Clipping [3 Marks]

- Build a fresh model starting from the same wide initialization as Task 5.1.
- Stabilize the same unstable starting point rather than using a different one.
- Immediately overwrite the encoder's W_{hh} with orthogonal initialization.
- Add gradient clipping with maximum norm 1.0 before every optimizer step.
- Train for 30 epochs.
- Plot this run's gradient norm overlaid against Task 5.1's curve.
- Use the same logarithmic axes.
- Keep this model in memory or checkpoint it.
- This is the model you will evaluate in Task 6.

Task 6: Long-Term Dependency Evaluation [8 Marks]

Note: You can directly use the inbuilt BLEU score function from the ***`nltk`*** *library for this task.*

6.1 Length-Stratified BLEU [3 Marks]

- Use the fully stabilized model from Task 5.2.
- The model should use orthogonal initialization and gradient clipping.
- The model should be trained for 30 epochs.
- Use greedy decoding.
- Compute smoothed Corpus BLEU separately for: short; medium; long.
- Report the three values in a table.
- Reference range:
  - short $\approx$ 0.9–0.95;
  - medium $\approx$ 0.25–0.35;
  - long $\approx$ 0.15–0.20.

6.2 Qualitative Evaluation [3 Marks]

- Run greedy inference on the exact test examples below.
- Display the source, ground truth, and model prediction.
- The prediction must be generated by the trained model.

Short
Source: the old girl chases the red ball
Ground Truth: redai balex oldai girol chaun

Medium
Source: the old wizard throws the brave table and the blue fox finds the brave ball
Ground Truth: braai balex bluai foxol finun yol braai tabex oldai wizol thrun

Long
Source: the old dog finds the blue book and the swift cat sees the blue mountain and the small robot follows the old lamp and the young dog greets the quiet cloud
Ground Truth: quiai cloex youai dogol greun yol oldai lamex smaai robol folun yol bluai mouex swiai catol seeun yol bluai booex oldai dogol finun

6.3 Long-Term Dependency Gap [2 Marks]

Using your BLEU results, compute:

$$\Delta_{\text{short}-\text{long}} = \text{BLEU}_{\text{short}} - \text{BLEU}_{\text{long}}$$

and

$$R_{\text{long}} = \frac{\text{BLEU}_{\text{long}}}{\text{BLEU}_{\text{short}} + \epsilon}$$

Here:

- $\text{BLEU}_{\text{short}}$ is the corpus BLEU score for the short bucket.
- $\text{BLEU}_{\text{long}}$ is the corpus BLEU score for the long bucket.
- $\Delta_{\text{short}-\text{long}}$ is their absolute gap.
- R_{long} is the long-bucket BLEU as a fraction of the short-bucket BLEU.
- ϵ is a small constant, e.g. 10^{-8}, added only to avoid division by zero.

Report both values.